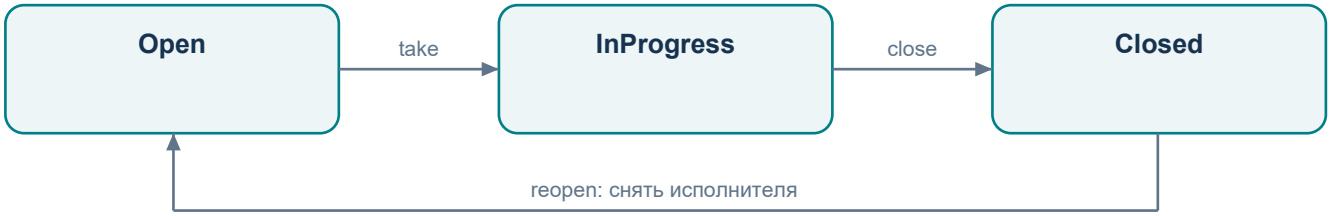
HelpDesk API

Учимся программировать
на готовом проекте

Учебник с нуля: от переменной до API

Простые объяснения, первые запросы, чтение C# по шагам, упражнения с ответами и выбор инструментов.



C# / .NET 10 / ASP.NET Core Web API
EF Core / Microsoft SQL Server / xUnit

Портфолио-проект · Русская версия · 10 ноября 2026

Содержание

1. Как учиться по этому проекту 3

2. Что мы создаём и зачем 4

3. Папки, файлы и команды 5

4. Первые понятия C# 6

5. HTTP и JSON: разговор с программой 8

6. Первый запуск и первая заявка 9

7. Следим за созданием в коде 10

8. Таблицы, SQL Server и EF Core 11

9. Вход и права 13

10. Правила, версии и ошибки 14

11. Списки и страницы 15

12. Тесты: проверяем ожидания 16

13. Остальные инструменты простыми словами 17

14. Практика и справка по чтению 18

15. Что уже проверено и чему учиться дальше 19

1. Как учиться по этому проекту

Это руководство начинается с основных понятий. Знать C#, базы данных и веб-разработку заранее не требуется. Мы будем изучать готовый HelpDesk небольшими частями: сначала увидим задачу, затем разберём язык и запросы, после этого проследим работу программы.

Читайте одну главу за занятие. Открывайте названный файл, находите конкретный метод и пробуйте объяснить его своими словами. Не нужно сразу понимать весь файл. Короткие примеры кода в тексте служат для объяснения: заменять ими готовую программу не нужно.

После этого руководства вы сможете запустить проект, отправить запрос, найти отвечающий за него код и сделать небольшое изменение. Один проект не заменяет изучение всего C# и администрирования серверов, но даёт основу, на которую можно опираться дальше.

Порядок занятий

1. Главы 2-4: задача, файлы и основы языка.
2. Главы 5-6: запросы и первая работа с приложением.
3. Главы 7-11: чтение настоящего кода, база, вход и правила.
4. Главы 12-15: тесты, остальные инструменты и упражнения.

Если встретили ошибку команды, остановитесь и прочитайте первую ошибку. Не запускайте следующие шаги вслепую. Если не получается объяснить пример, вернитесь на один раздел назад. Несколько вечеров на одну тему - нормальный темп.

Прежний подробный текст сохранён в docs/technical-reference.md. Он пригодится позднее как технический справочник. Основной учебный маршрут находится здесь.

2. Что мы создаём и зачем

Представьте компанию: у сотрудника перестал работать принтер. Он создаёт заявку «Принтер не печатает», описывает проблему и выбирает срочность. Поддержка берёт обращение, обсуждает детали в комментариях и закрывает после решения.

Наш HelpDesk выполняет эту задачу. Обычный пользователь видит свои обращения. Сотрудник поддержки видит все. Пользователь поддержки отличается наличием роли Support - группы разрешённых действий.

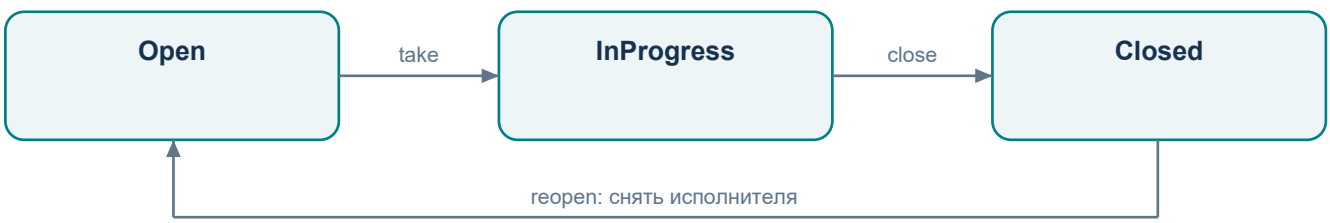
Что содержит заявка

Поле	Простой смысл
Title	Короткое название
Description	Подробности проблемы
Priority	Срочность: Low, Normal, High, Critical
Status	Открыта, в работе или закрыта
OwnerId	Кто создал заявку
AssigneeId	Кто из поддержки её выполняет
CreatedAt / UpdatedAt	Время создания / последнего изменения
ClosedAt	Время закрытия, если уже закрыта
Id	Уникальный номер заявки
Version	Метка состояния, защищающая от старых правок

Id имеет тип GUID: это длинный уникальный идентификатор. Его знание само по себе не разрешает читать заявку. Права проверяются отдельно.

Состояния

Open - заявка открыта. InProgress - сотрудник взял её на себя. Closed - работа закончена. Закрытое обращение нельзя редактировать и комментировать до повторного открытия. При открытии заново прежний исполнитель снимается.



Взять заявку и закрыть назначенную себе может только Support. Владелец может открыть свою закрытую заявку снова. Отдельный сайт не создавался: для знакомства мы используем Swagger UI, страницу отправки запросов к API.

Клиент, сервер и база

Клиент отправляет запрос. Серверная часть программы, или backend, выполняет работу. API - правила обращения к этой программе: адрес, отправляемые поля и ожидаемый ответ. Слово «сервер» здесь может означать программу на вашем компьютере, покупать отдельную машину не требуется.

База данных хранит записи между запусками. SQL Server - программа управления базой. Закрытие API не должно удалять заявки: они находятся в базе. Но удаление её хранилища удалит и записи.

Первое упражнение

Придумайте название, описание и приоритет проблемы. Пример: «VPN недоступен», «После смены пароля соединение отклоняется», High. Сохраните свой пример: позже вы отправите его в приложение.

3. Папки, файлы и команды

Корень проекта - папка с README.md и HelpDesk.sln. Открывайте терминал именно в ней. Терминал - окно ввода текстовых команд; здесь используется PowerShell. Редактор кода нужен для просмотра и изменения файлов. Сначала достаточно уже установленного редактора.

Путь от корня	Содержание
src/HelpDesk.Api	Приложение
Domain внутри приложения	Заявка и её правила
Application внутри приложения	Сценарии и данные запросов
Controllers внутри приложения	Приём HTTP-запросов
Infrastructure внутри приложения	База и учётные записи
tests	Автоматические проверки
scripts	Вспомогательные скрипты
docs	Текстовые объяснения
output/pdf	Руководство PDF

Файл .cs содержит C#-код. .csproj описывает проект и нужные библиотеки. .sln объединяет приложение и тесты. JSON и YAML - форматы текстовых данных и настроек. Папки bin и obj создаёт сборка: для первого чтения их пропускайте.

Как читать команду

```
dotnet build HelpDesk.sln -c Release
```

dotnet - инструмент, build - действие «собрать», HelpDesk.sln - что собираем, -c Release - настройка сборки. Сборка переводит исходный код в пригодный для выполнения вид. Она ещё не запускает API.

restore загружает библиотеки, build собирает код, run запускает приложение, test выполняет проверки, publish готовит файлы для отдельного запуска. Пока достаточно различать эти действия. Не вводите поясняющие предложения в терминал.

Упражнение

Найдите src/HelpDesk.Api/Domain/Ticket.cs и tests/HelpDesk.Api.Tests/TicketTests.cs. Первый описывает заявку, второй проверяет её поведение. Откройте их рядом, пока не пытаясь понять все строки.

4. Первые понятия C#

Код этой главы - примеры языка, а не команды PowerShell. Не вставляйте его в терминал.

Значения, переменные и типы

```
string title = "VPN не работает";
int count = 3;
bool isSupport = true;
```

Переменная хранит значение под именем. title - текст, count - целое число, isSupport - «да» или «нет». string, int и bool называются типами. Тип помогает заметить неверное использование данных ещё до запуска. Точка с запятой завершает такую инструкцию; кавычки обозначают текст.

var означает, что компилятор определит тип по правой части. После var count = 3 переменная всё равно имеет целочисленный тип; она не становится «чем угодно».

Условия

```
if (Status == TicketStatus.Closed)
{
    throw new RuleException("Reopen the closed ticket first.");
}
```

if означает «если». == сравнивает значения, а = записывает значение. Фигурные скобки объединяют действия. throw прерывает обычное выполнение и сообщает об ошибке. RuleException - название ошибки нашего проекта. Общий обработчик позже превратит её в понятный HTTP-ответ.

Класс, объект и свойство

Класс описывает устройство объектов одного вида. Ticket - описание заявки. Конкретный объект Ticket - одна заявка со своим названием и номером. Свойство - именованная часть данных объекта.

```
public string Title { get; private set; } = "";
```

public разрешает читать Title из другого кода. get означает чтение. private set разрешает изменять это свойство только внутри Ticket. Так другой код должен вызвать предусмотренный метод, где выполняются проверки. Пустая строка здесь - начальное значение; создание заявки проверит настоящий текст.

Метод, параметр и результат

Метод - именованное действие. Вызов ticket.Reopen(now) означает: «у этой заявки выполнить Reopen, передав время now». Переданное значение называют аргументом, а описание принимаемого значения - параметром. return возвращает результат.

Откройте Ticket.cs и прочитайте только Reopen: проверка, новый статус, очистка исполнителя и даты, затем Touch. null означает отсутствие значения, например отсутствие исполнителя. ? у типа показывает, что отсутствие допустимо.

enum задаёт набор вариантов, например TicketStatus. Коллекция содержит несколько элементов, например список заявок. Task представляет операцию, которая может закончиться позже. async/await позволяют ожидать её, например ответ базы. CancellationToken передаёт отмену. Эти три механизма подробно изучите после основных сценариев.

Проверка понимания

1. Один знак = записывает значение, два == сравнивают.
2. private set у Title не позволяет менять название из любого места в обход правил.
3. AssigneeId = null снимает исполнителя.
4. Найдите Close: он требует InProgress и совпадения исполнителя с закрывающим сотрудником.

5. HTTP и JSON: разговор с программой

HTTP - правила обмена запросами и ответами. Запрос содержит действие, адрес, заголовки и иногда тело с данными. Заголовки - служебные сведения. JSON - текстовый формат записи данных.

```
POST /api/tickets
Content-Type: application/json

{"title":"VPN недоступен","description":"Ошибка входа","priority":"High"}
```

POST здесь означает создание. /api/tickets - путь внутри API. Content-Type сообщает формат тела. В JSON фигурные скобки содержат объект, имена полей и текстовые значения записаны в двойных кавычках, запятые разделяют поля. Для настоящего создания нужен также токен входа; сейчас мы рассматриваем форму запроса.

Полный адрес выглядит как http://localhost:5080/api/tickets. localhost означает ваш компьютер, 5080 - порт, то есть номер входа для программы. GET читает сведения, POST создаёт запись или выполняет команду, PUT изменяет доступные поля.

Код ответа	Значение	Действие пользователя
200	Успех	Посмотреть результат
201	Создана запись	Сохранить id и version
400	Неверный запрос	Исправить поля или JSON
401	Нет действительного входа	Войти и передать токен
403	Не хватает прав	Проверить роль
404	Доступная заявка не найдена	Проверить id и пользователя
409	Конфликт состояния или версии	Прочитать detail, обновить данные
429	Слишком частые запросы входа	Подождать
500	Неожиданная ошибка	Сохранить traceId для диагностики

Чужая заявка тоже возвращает 404, чтобы не раскрывать её существование. Чтение и изменение - разные действия: GET не должен менять заявку.

Упражнение с ответом

Создание без токена должно дать 401. Редактирование закрытой заявки с правильным входом и свежей версией - 409. Причины отказа разные, поэтому коды тоже разные.

6. Первый запуск и первая заявка

Для рекомендуемого способа нужен Docker Desktop с Linux containers и Compose v2. SQL Server в контейнере требует x86-64 окружения; выделите Docker минимум 4 ГБ памяти. Сначала установите и запустите Docker, если его ещё нет. В исходной среде Docker и SQL Server отсутствовали: настоящий контейнерный запуск здесь не проверялся.

Создаём локальные настройки

В корне проекта откройте PowerShell и выполните:

```
powershell -ExecutionPolicy Bypass -File scripts/New-LocalSettings.ps1
```

Скрипт создаёт .env со случайными паролями. ExecutionPolicy Bypass относится к этому запуску скрипта. Если .env уже существует, скрипт отказывается переписывать его. Это ожидаемо: используйте существующий файл. Не публикуйте его содержимое.

Запускаем API и базу

```
docker compose up --build -d
docker compose ps
```

up запускает окружение; --build собирает образ API; -d оставляет его работать в фоне. ps показывает состояние. Первый запуск скачивает образы и может занять несколько минут. Откройте http://localhost:5080/health/ready. После успешного ответа откройте http://localhost:5080/swagger.

Входим как Alice

В Swagger раскройте POST /api/auth/login и нажмите Try it out. Введите email alice@example.test. Пароль возьмите из HELPDESK_DEMO_PASSWORD в вашем .env. SQL_SA_PASSWORD - другой пароль, предназначенный базе. useCookies оставьте false. Нажмите Execute.

В ответе найдите accessToken: временное подтверждение входа. Скопируйте значение, нажмите Authorize и вставьте только токен, без кавычек и слова Bearer. Теперь GET /api/tickets должен показать заявки Alice.

Создаём и обрабатываем обращение

Отправьте придуманный пример через POST /api/tickets. Сохраните id и version из ответа. id выбирает заявку; version показывает состояние, которое вы меняете. После каждого успешного изменения берите новую version из ответа.

Очистите авторизацию кнопкой Logout в окне Authorize. Войдите как support@example.test с тем же демопаролем и вставьте новый accessToken. Выполните take, добавьте комментарий, затем close. Каждый раз копируйте свежую версию. Обычный пользователь не выполняет take и close.

Вернитесь к Alice: изменение закрытой заявки даст 409. Выполните reopen, затем редактирование с новой версией. Войдите как bob@example.test и прочитайте заявку Alice: ожидается 404.

Останавливаем окружение

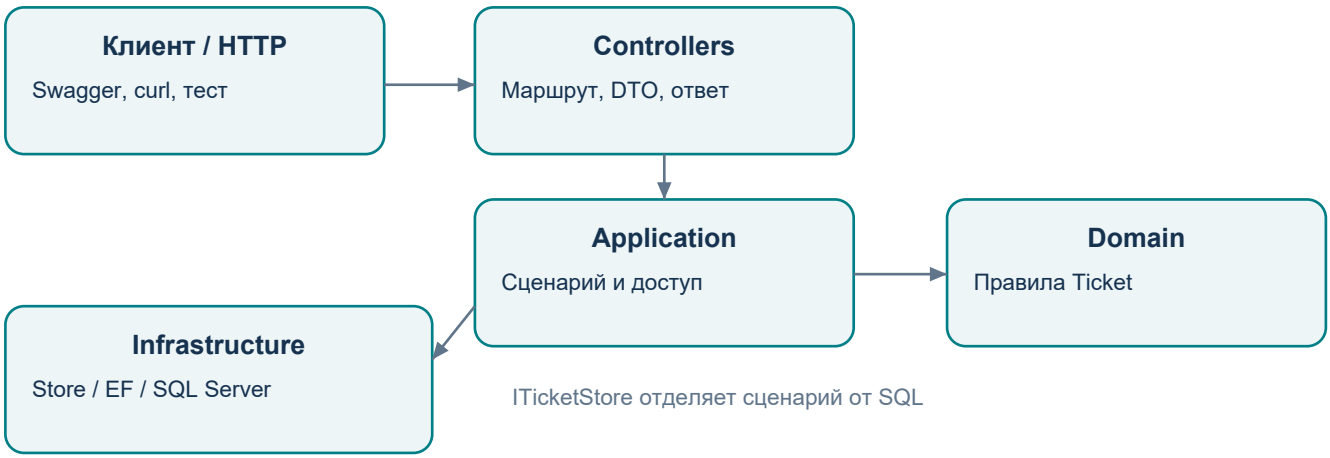
```
docker compose down
```

Данные сохранятся. Не добавляйте -v для обычной остановки: он удаляет хранилища базы и ключей. Смена .env сама не меняет уже созданные пароли. Если запуск не удался, смотрите docker compose logs api db. Если docker не найден, проверьте установку. При 401 проверьте токен, при 409 - статус и версию.

Запуск без Docker и подробная диагностика сохранены в docs/maintainer-readme.md. Это отдельный маршрут для компьютера с установленным SQL Server. Обычный dotnet run не читает .env автоматически.

7. Следим за созданием в коде

Теперь проследим один POST. Названные далее папки находятся внутри src/HelpDesk.Api. Не читайте весь проект: переходите только между указанными методами.



Приём запроса

В Controllers/TicketsController.cs найдите Create. [HttpPost] указывает метод запроса. Общий путь контроллера - api/tickets. [Authorize] требует вход.

CreateTicketRequest описывает разрешённые входные поля. Такой объект передачи данных называют DTO. Найдите его в Application/Contracts.cs: там Title, Description и Priority, но нет OwnerId. Владелец берётся из проверенного входа, поэтому клиент не может выбрать чужого владельца через JSON.

Сценарий и правила

В Application/TicketService.cs найдите CreateAsync. Он вызывает Ticket.Create, затем store.Add и store.SaveAsync. Сервис организует действия, но не пишет SQL-команду INSERT.

В Domain/Ticket.cs найдите Create и Edit. Проверяются название, описание и приоритет. Trim убирает пробелы по краям. Touch обновляет время и Version. Создание объекта в памяти ещё не означает сохранение в базе.

Сохранение и ответ

В Infrastructure/EfTicketStore.cs Add передаёт объект EF Core. SaveAsync вызывает SaveChangesAsync - здесь изменения отправляются в базу. TicketDto.From формирует данные ответа. Контроллер возвращает 201 и Location, адрес новой заявки.

Задача разделения - не смешивать HTTP, правила и SQL. Альтернатива - весь код в контроллере: одинаковые проверки будут повторяться. Другой вариант - много отдельных проектов и посредников. Для небольшой программы выбраны одна сборка и понятные папки.

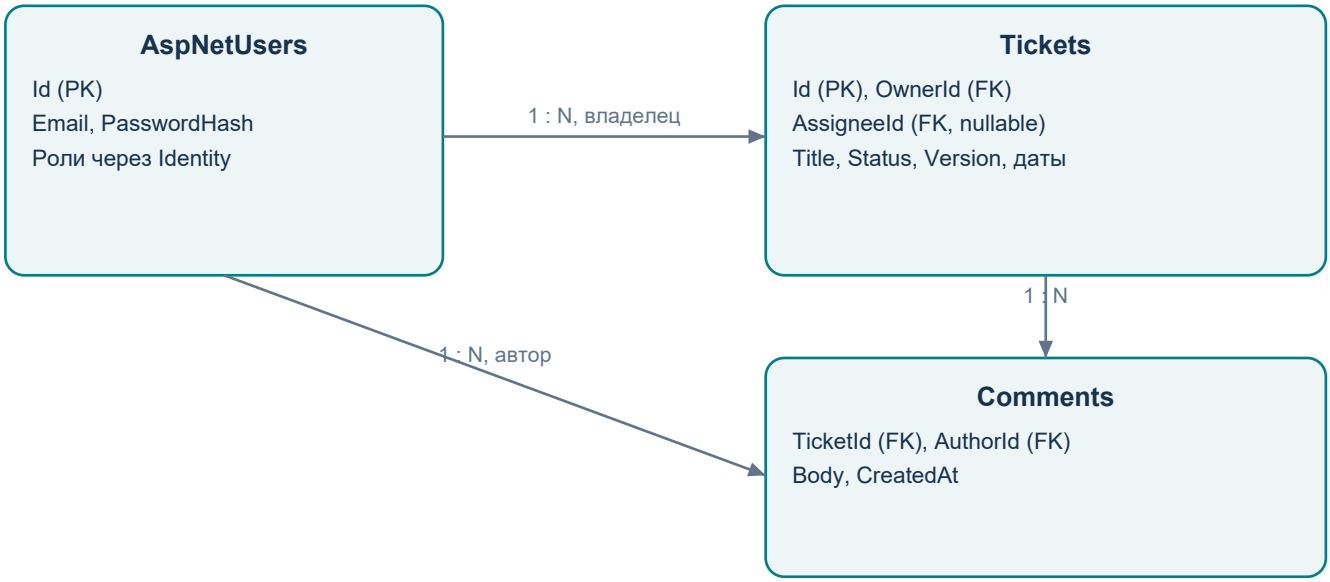
Упражнение

Запишите путь без подсказки. Ответ: TicketsController.Create -> TicketService.CreateAsync -> Ticket.Create -> EfTicketStore.SaveAsync. Объясните по одному предложению про каждое место.

8. Таблицы, SQL Server и EF Core

Таблица похожа на таблицу в тетради: столбцы обозначают виды сведений, строки - отдельные записи. В Tickets строка соответствует заявке, в Comments - комментарию, в AspNetUsers - пользователю.

Первичный ключ - собственный номер записи. Внешний ключ - ссылка на другую запись. У комментария TicketId указывает заявку, AuthorId - автора. База проверяет, что связанная запись существует.



SQL Server

Задача: хранить записи между запусками, проверять связи и совместно сохранять связанные изменения. SQL Server выбран по заданию. Альтернативы - PostgreSQL, MySQL, SQLite. Без системы управления базой можно сохранять JSON-файлы, но самостоятельно решать одновременную запись и восстановление после ошибок.

SQL - язык команд для реляционной базы. SELECT читает, INSERT добавляет, UPDATE изменяет. Ниже учебный пример, для запуска его выполнять не нужно:

```
SELECT Title FROM Tickets WHERE Status = 0;
```

В нашем enum Open равен 0. В таблице сохраняется число, а API показывает человеку слово Open.

EF Core

Задача: связать объекты C# с таблицами. EF Core умеет переводить многие операции в SQL. Такое средство называют ORM. DbContext - объект работы с базой, DbSet - набор записей определённого вида.

Причина выбора: заданный стек и совместимость с Identity и миграциями. Альтернативы - Dapper или ручные команды ADO.NET. Без EF пришлось бы писать INSERT/UPDATE, передавать параметры и вручную читать столбцы результата. Понимать SQL всё равно полезно.

Миграции

Миграция - сохранённая инструкция изменения структуры базы. InitialCreate создаёт начальные таблицы. Найдите её в Infrastructure/Migrations и прочитайте создание Tickets и Comments. Длинный Designer пока можно пропустить.

Задача миграций - одинаково обновлять разные базы. dotnet-ef - инструмент создания и применения этих инструкций. Альтернатива - собственные SQL-скрипты с учётом применения. Без этого нужно вручную помнить, какие столбцы уже существуют на каждом компьютере.

Созданную миграцию нужно читать: она может удалить данные. Для новой функции добавляют новую миграцию, а не переписывают уже применённую. В таблице истории EF отмечает выполненные изменения.

9. Вход и права

Регистрация создаёт пользователя. Вход проверяет email и пароль. Аутентификация - установление личности. Авторизация - решение, что этой личности разрешено.

Identity

Задача: использовать готовую систему учётных записей вместо собственной разработки паролей и токенов. Identity выбран по заданию. Он хранит специальный хеш для проверки пароля, а не открытый пароль.

Альтернативы - внешний сервер входа, например Keycloak или Microsoft Entra ID. Без готового решения нужно самостоятельно разрабатывать безопасное хранение паролей, выдачу токенов и ограничения входа. Это отдельная сложная область.

Здесь пароль содержит минимум 12 символов, буквы разных регистров, цифру и спецсимвол. Пять неверных попыток блокируют вход на пять минут. Настоящая доставка писем для восстановления не настроена.

Токены и роли

accessToken передаётся в следующих запросах и действует 30 минут. refreshToken позволяет получить новую пару и действует 7 дней. Здесь токены Identity, не JWT; разбирать их вручную не нужно. Logout в Swagger убирает токен из интерфейса, но не отзывает выданную копию на сервере.

Support нельзя получить, отправив role при регистрации. Выдача роли выполняется доверенной консольной командой из подробной инструкции запуска. После выдачи роли нужно войти заново.

Проверка конкретной заявки

Bob может войти, но не читать заявку Alice. В EfTicketStore найдите Visible: обычному пользователю добавляется условие OwnerId == actor.Id. Support видит все строки. Actor содержит номер и роль из проверенного входа, а не из произвольного JSON.

Комментарии сначала проверяют доступ к родительской заявке. Иначе чужие сведения можно было бы получить через них. Знание GUID и наличие любого токена такую проверку не заменяют.

Упражнение с ответом

Без токена - 401. Обычному пользователю на take - 403. Bob на GET заявки Alice получает 404. Это три разные причины: нет входа, нет роли, нет доступного объекта.

10. Правила, версии и ошибки

Название не должно быть пустым, приоритет должен быть разрешённым, закрытая заявка доступна только для чтения. Это бизнес-правила: требования самой задачи поддержки.

В Contracts.cs атрибуты Required, StringLength и Range проверяют поля запроса. В Ticket.cs проверки защищают сущность даже при вызове без HTTP, например при создании демоданных. Альтернатива атрибутам - ручные проверки или отдельный валидатор. Без общего подхода одинаковые условия и ошибки пришлось бы повторять.

Защита от старой правки

Alice и Support открыли одну заявку. Support изменил её первым. Если принять старую форму Alice без проверки, его правка может потеряться. Поэтому запрос изменения передаёт Version.

Сервер сравнивает её с текущей и возвращает 409 при несовпадении. Получите свежую заявку, изучите изменения и повторите свою правку осознанно. Комментарий тоже обновляет версию.

Есть также проверка в базе для двух запросов, которые прочитали одну версию почти одновременно. В HelpDeskDbContext найдите IsConcurrencyToken. EF учитывает исходную версию при UPDATE, и второй писатель получает конфликт. Это называют оптимистической проверкой одновременных изменений.

Альтернативы - SQL Server rowversion, HTTP ETag или блокировки. Без защиты последняя запись могла бы затереть предыдущую. GUID версии нельзя сравнивать как дату или последовательный счётчик.

Читаем ошибку

Формат Problem Details содержит status - код, title - короткое название, detail - объяснение, traceId - метку для поиска в журнале. Неверные поля могут быть перечислены в errors. ApiExceptionHandler превращает внутренние ошибки в общий формат. Без него похожие try/catch пришлось бы писать в каждом контроллере.

Сначала читайте код и detail. 409 может означать старую версию или запрещённый переход статуса. Повторять тот же запрос без изменения причины бесполезно.

11. Списки и страницы

Страница - часть большого списка. page=1 выбирает первую страницу; pageSize=20 просит до двадцати записей. Максимальный размер здесь 100.

```
/api/tickets?status=Open&priority=High&page=1&pageSize=20
```

После ? идут настройки, разделённые &. Здесь нужны одновременно открытые заявки и высокий приоритет. В ответе items - записи, totalCount - общее количество подходящих, totalPages - число страниц.

sortBy выбирает CreatedAt, UpdatedAt или Priority. descending=true означает обратный порядок, например новые даты сначала. При равенстве используется дополнительная сортировка по Id.

LINQ

В EfTicketStore.ListAsync сначала Visible, затем фильтры, подсчёт и сортировка. Where отбирает, OrderBy упорядочивает, Skip пропускает, Take ограничивает. ToListAsync получает результат.

Эти операции относятся к LINQ, способу описывать выборки в C#. Пока мы работаем с IQueryable, EF строит описание запроса. Альтернатива - вручную написать SQL. Без отбора в базе пришлось бы сначала читать лишние записи в память. Проверка владельца применяется до выдачи данных.

Упражнение

Для page=3 и pageSize=10 нужно пропустить 20 записей и взять до 10. Формула: (page - 1) * pageSize. Если записей всего 23, на третьей странице будет 3. Пустой список не является ошибкой.

12. Тесты: проверяем ожидания

Тест выполняет действие и сравнивает результат с ожиданием. Так проще заметить, что изменение сломало прежнее правило. Ручная проверка Swagger полезна, но не заменяет повторяемые автоматические проверки.

xUnit выбран по заданию. Альтернативы - NUnit и MSTest. Без тестового инструмента нужно вручную организовать запуск и отчёт. Fact отмечает сценарий, Theory с InlineData повторяет его для разных входов.

Читаем тест

В TicketTests найдите Create_SetsDefaultsAndTrimsText. Сначала создаётся заявка, затем Assert.Equal сравнивает значения. Assert.Null ожидает отсутствие значения, Assert.Throws - определённую ошибку. Здесь достаточно объекта Ticket без базы.

ApiTests проверяет совместную работу HTTP, входа и сохранения. Такой тест называют интеграционным. WebApplicationFactory запускает приложение для проверок без ручного управления сервером. SQLite даёт отдельную временную базу в памяти каждому тесту. Альтернатива - каждый раз отдельно запускать API и SQL Server.

SQLite отличается от SQL Server. Его успешный прогон не доказывает все особенности настоящей базы. Для тех же тестов на SQL Server подготовлен отдельный job GitHub Actions. Локально выполнен 41 случай на SQLite, все прошли. Процент покрытия не измерялся.

Запускаем проверки

Нужен SDK .NET 10. Из корня проекта:

```
dotnet restore HelpDesk.sln --locked-mode
dotnet build HelpDesk.sln -c Release --no-restore
dotnet test --solution HelpDesk.sln -c Release --no-build
```

Сначала загружаются библиотеки, затем собирается код, затем выполняются тесты. Параметры --no-restore и --no-build пропускают уже сделанные шаги. Тестовый раннер Microsoft.Testing.Platform выбран в global.json; вручную настраивать его для первого запуска не нужно.

Упражнение

Найдите OtherUsersTicket_IsHiddenOnEveryAccessibleRoute и прочитайте InlineData. Проверяются чтение, изменение, чтение и добавление комментариев, повторное открытие. Объясните, почему защищать только GET заявки недостаточно.

13. Остальные инструменты простыми словами

Язык и платформа

C# записывает инструкции. Он выбран по заданию и работает вместе с остальными библиотеками. Альтернативы - Java, Python, Go. Без C# понадобился бы другой язык с теми же проверками.

.NET выполняет код и предоставляет общие средства. Версия 10 выбрана по заданию. SDK собирает и тестирует, runtime запускает готовое. Альтернатива - другая платформа или версия. Без SDK пришлось бы отдельно вызывать компилятор и собирать нужные файлы.

ASP.NET Core принимает HTTP и выбирает обработчики. Контроллеры делают места входа видимыми. Альтернатива - Minimal APIs в том же ASP.NET Core или другой веб-фреймворк. Без фреймворка нужно вручную разбирать запросы и связывать адреса с действиями.

Зависимости и настройки

TicketService получает store и clock через конструктор. Это внедрение зависимостей: класс явно сообщает, что ему нужно. Program.cs связывает ITicketStore с EfTicketStore. Интерфейс описывает действия, конкретный класс их выполняет. Альтернатива - ручное создание через new. Без общего механизма пришлось бы повторять создание и очистку объектов.

NuGet загружает библиотеки и их зависимости. Альтернатива - копировать DLL вручную и самому искать совместимые версии. packages.lock.json фиксирует выбранный набор. appsettings хранит обычные настройки, переменные окружения - значения запуска. user-secrets - вариант хранения локальных секретов вне кода. Без этого пароль легко оказался бы прямо в исходнике.

Демонстрация и запуск

Swagger UI показывает API и выполняет запросы. OpenAPI описывает его для программ; Swashbuckle строит описание по коду. Альтернативы - Scalar, NSwag, curl или Postman. Без генератора пришлось бы вручную поддерживать описание всех маршрутов. docs/requests.http содержит готовые примеры для HTTP-клиента редактора.

Docker упаковывает среду, Compose запускает API вместе с SQL Server. Контейнер - отдельная среда запуска, volume - её сохраняемое хранилище. Альтернатива - ручная установка программ. Без Compose нужно отдельно задавать сеть, настройки, хранение и ожидание готовности базы.

Git хранит историю изменений, GitHub размещает её, GitHub Actions автоматически собирает и проверяет проект. Альтернативы - GitLab CI и Azure Pipelines. Без автоматизации команды приходится повторять вручную. Workflow подготовлен, но здесь не запускался на GitHub.

Вспомогательные средства

TimeProvider передаёт время так, чтобы его можно было заменить в тесте. Альтернатива - везде вызывать DateTime.UtcNow. Без общих часов сложнее повторять временные сценарии. Даты API возвращаются в UTC с буквой Z, это единая точка отсчёта.

Data Protection защищает токены, его ключи сохраняются отдельно. Без сохранения ключей пересоздание контейнера могло бы нарушить проверку прежних токенов. Ограничитель частоты защищает вход от слишком частых запросов. Журнал помогает искать ошибки, health-маршруты показывают состояние процесса и чтения базы.

PDF собирается Python-скриптом через ReportLab. Poppler создаёт изображения страниц для проверки вёрстки. Альтернатива - Word с экспортом PDF. Без скрипта оглавление и оформление пришлось бы обновлять вручную. Python не требуется для запуска API.

14. Практика и справка по чтению

Маршрут первого самостоятельного прохода

1. Ticket.cs: прочитайте только Reopen и объясните каждую строку.
2. Contracts.cs: сравните CreateTicketRequest и TicketDto. Что клиент присылает, что получает?
3. TicketsController.Create: найдите маршрут и отправку результата.
4. TicketService.CreateAsync: перечислите действия по порядку.
5. EfTicketStore.SaveAsync: найдите момент сохранения.
6. EfTicketStore.Visible: объясните запрет чужих заявок.
7. TicketTests и ApiTests: найдите проверки этих правил.
8. Program.cs и Compose читайте после основ: это подключение и запуск частей.

Короткая карта находится также в docs/reading-map.md. Сгенерированные Designer, ModelSnapshot и содержимое bin/obj в первом проходе пропускайте.

Первое изменение

В TicketTests добавьте тест: создать открытую заявку, вызвать Reopen и ожидать RuleException. Используйте соседний Assert.Throws как образец. Тест должен пройти, потому что открыть повторно разрешено только Closed. Миграция не нужна: структура базы не меняется.

Второе изменение

В DatabaseSetup найдите текст демозаявки и измените его. Соберите проект. Существующая непустая база не наполнится заново: старый текст сам не изменится. Сначала объясните эту причину, не удаляя данные. Для просмотра эффекта позже можно создать отдельную чистую учебную базу.

После основных уроков

Добавьте Category. Сначала решите допустимые значения и значение для старых записей. Затем меняйте Ticket, входные DTO, TicketDto.From и настройку EF. Создайте новую миграцию, прочитайте её, добавьте тест сохранения и чтения. Это сложное упражнение через все слои, начинать обучение с него не нужно.

В отладчике точка остановки приостанавливает выполнение. Начните с TicketsController.Create и посмотрите request и Actor при POST. Затем переходите в CreateAsync. Способ установки точки зависит от редактора. Не публикуйте пароль или токен на снимках.

15. Что уже проверено и чему учиться дальше

Приложение собрано и опубликовано, 41 тест прошёл на SQLite. Опубликованная сборка отдаёт Swagger и отклоняет анонимный доступ. SQL Server и Docker отсутствовали: настоящий контейнерный прогон остаётся отдельной проверкой. Результаты описаны в docs/verification.md.

Для публичного сервера нужны отдельные настройки HTTPS, доступа к базе, ключей и резервного копирования. Локальная демонстрация включает демоданные и удобные настройки запуска. Их не следует переносить без изменений на рабочий сервер. Здесь нет настоящей доставки email, мгновенного отзыва access-токенов, вложений, истории статусов и SLA.

Дальше по C# изучайте коллекции, интерфейсы, ошибки и async/await. По HTTP - заголовки и коды, по SQL - SELECT, JOIN, ключи и транзакции. Затем возвращайтесь к EF, миграциям и авторизации. Используйте четыре вопроса: что пришло, кто обращается, какое правило действует, где сохраняется результат.

Словарь

Термин	Значение
Backend	Серверная обработка запросов и данных
API	Правила обращения к программе
JSON	Текстовая запись структурированных данных
DTO	Объект входных или выходных полей
Сущность	Внутренний объект с данными и правилами
Метод	Именованное действие в коде
Библиотека	Готовый код для использования
Миграция	Изменение структуры базы
Токен	Временное подтверждение входа
Роль	Группа разрешённых действий
Тест	Автоматическая проверка ожидания

Материалы для следующего этапа

Сложные детали сохранены в docs/technical-reference.md, точные поля и маршруты - в docs/api.md, подробный запуск - в docs/maintainer-readme.md. Читать всё в первый день не нужно.

- Миграции EF Core.
- Вход через Identity.
- xUnit и тестовый раннер.

Сначала наблюдайте поведение на собственном примере, затем читайте объяснение. Умение предсказать ответ, найти правило и подтвердить его тестом важнее запоминания всех названий.